

Loss and optimization

A loss turns prediction error into a number; a gradient estimates how parameter changes would move that number.

15–20 MINUTES

Prerequisite: training vs inference

INTERACTIVE LOSS

1 • Forward

Produce a prediction.

2 • Loss

Measure disagreement.

3 • Gradient

Estimate a downhill direction.

4 • Update

Take a controlled step.

Why training needs an objective

“Make better predictions” must become something the optimizer can compare. A **loss function** assigns a numeric penalty to a prediction and target. Lower loss means better according to that chosen rule. It does not automatically mean safer, fairer, or more useful in deployment.

For a toy numeric prediction, squared error is $(\text{prediction} - \text{target})^2$. Detection training uses specialized classification and localization terms, but the role is the same: turn discrepancies into a differentiable objective.

Interactive squared error

Target = 3. Move the prediction.



Loss: **9.00**

The curve is lowest at the target. Farther predictions receive a larger penalty.

Central mental model

One training step is: predict → measure loss → compute gradients → update parameters. Repeating this over batches changes the model.

Gradient descent without the calculus derivation

The **gradient** describes how a small increase in each parameter would change the loss. Moving a parameter in the opposite direction tends to reduce loss locally. The **learning rate** controls the step size.

new parameter = old parameter – learning rate × gradient

A very large rate can jump past useful values or make training unstable. A very small rate can make progress slow. The gradient is local information, so one update does not guarantee the globally best parameters.

Batches and epochs

A **batch** is the set of examples used for one update. An **epoch** is one pass through the training set. Training over multiple epochs lets the model revisit examples as its parameters change. Random batches make gradients noisy, but that noise can be useful and makes large datasets practical.

Training loss alone is not enough. A model can reduce loss by fitting peculiarities of the training examples. The next lesson separates this fitting signal from performance on unseen data.

Applied to our cubes

REPOSITORY OBSERVATION The documented training run used batches of 16 for 30 epochs and selected an epoch-18 checkpoint under a validation rule.

GENERAL PRINCIPLE An epoch count and learning rate are experiment settings. Their presence does not prove convergence or generalization; curves and held-out results supply that evidence.

PRACTICAL IMPLICATION When fine-tuning, change a justified factor, keep comparable evaluation data, and retain the old checkpoint for rollback.

Project anchor: training record.

Check your understanding

What does the learning rate directly control?

- ☐ Number of target labels
- ☐ Meaning of camera pixels
- ☐ Size of parameter updates

Explain it back

Explain one training step without saying that the model “just learns.” Include what the loss measures, what the gradient provides, and what the learning rate controls.

Primary sources

- Deep Learning, Chapter 8 — optimization for deep models.
- D2L: Linear regression from scratch — a small complete training loop.